recursion

mahmood

January 7, 2023

Definition 0.0.1. a **recursive** function is a function that calls itself

Example 0.0.1. consider factorial

```
def fac(n):
  if n == 1: return 1
  else: return fac(n-1) * n
```

1 base case

Definition 1.0.1. A recursive function definition has one or more base cases, meaning input(s) for which the function produces a result trivially (without recurring)

2 recurrence relation

to analyze the time complexity of recursive functions we use a mathematical concept called **recurrence relation**
a recurrence defines $T(n)$ in terms of T for smaller values
example: $T(n) = T(n-1) + 1$, here $T(n)$ is defined in terms of $T(n-1)$

2.1 initial condition

a recurrence relation must have **initial conditions**, initial conditions are values of the recurrence for small values of n
the values of $T(0), T(1)$ are usually sufficient as initial conditions

2.2 closed form

to solve a recurrence, we find a **closed form** for it, a closed form for $T(n)$ is an equation that defines $T(n)$ using an expression that does **not** involve T
there is no single method that works for all, we check for patterns and use substitution
to get a better understanding, we look at examples

Example 2.2.1. consider the following function that returns the factorial of a given number

```
def fac(n):
  if n == 1: return 1
  else: return n * fac(n-1)
print([fac(i) for i in range(1,10)])
```

for `fac(n)`, how many times is `fac` called?

$$T(n) = T(n-1) + C_1$$

C_1 represents the total time of all the operations that take constant time that the function does on every iteration
now this is obviously not what we wanna arrive at, we need to arrive at an expression that doesnt have T in it
we can substitute in $n-1$ because T is a function that takes any positive integer

$$T(n) = \underbrace{T(n-2) + C_1}_{T(n-1)} + C_1$$
$$T(n) = T(n-3) + 3C_1$$

now you probably already see the pattern here, on every iteration the function takes constant time C_1 to run, until it reaches the last iteration k at which point it we would've k times C_1, e.g.:

$$T(n) \leq T(n-k) + kC_1$$

now we know that the total number of times the function calls itself depends on the initial value of n, in this case that number would be $n-1$, as in, this function calls itself $n-1$ times
we substitute $k = n-1$ to get the total time this function takes to run

$$\begin{aligned} T(n) &= T(n-(n-1)) + (n-1)C_1 \\ &= T(1) + (n-1)C_1 \\ &= C_1 + (n-1)C_1 && \text{because } C_1 \geq T(1) \\ &= nC_1 \end{aligned}$$

so we conclude that

$$\Theta(n) = nC_1 \implies \Theta(n) = n$$

Example 2.2.2. consider the following function that returns the maximum value in a given array

```
def mymax(arr, l, r):
  if l == r:
    return arr[l]
  return max(arr[l], mymax(arr, l+1, r))
arr = [3, 10, 81, 30, 58, 0, -10]
print(mymax(arr, 0, len(arr)-1))
```

we need to find a recurrence for this function so we can analyze its time complexity
first thing we notice is on every iteration we have a constant time (excluding the recursive call), we call it C
we can also notice that the variable "r" never changes, so our recurrence should only depends on l
now we can guess that the recurrence equation is:

$$T(l) = T(l-1) + C$$

you might be confused (hopefully not) that we wrote $l-1$ even though in the recursive call its $l+1$, this is because we dont care about the value of the variable itself but rather about how many times the variable causes the function to call itself

now if we had written $l+1$ this would suggest that given the value l the function runs $l+1$ times which is not correct
and so this equals to $\Theta(n)$ (using the same proof as the previous problem)

Example 2.2.3. consider the following function that returns the fibonacci number at the given index

```
def fib(n):
  if n in [1, 2]: return 1
  else: return fib(n-1) + fib(n-2)
print([fib(i) for i in range(1,10)])
```

for `fib(n)`, how many times is `fib` called?

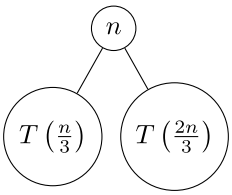
$$
\begin{aligned}
T(1) &= 1 \\
T(2) &= 1 \\
T(3) &= T(1) + T(2) + 1 = 1 + 1 + 1 = 3 \\
T(4) &= T(3) + T(2) + 1 = 3 + 1 + 1 = 5 \\
T(5) &= T(4) + T(3) + 1 = 5 + 3 + 1 = 9 \\
T(n) &= T(n-1) + T(n-2) + 1 = 2^n + 1
\end{aligned}
$$

3 recursion tree

Example 3.0.1. consider the following recurrence relation

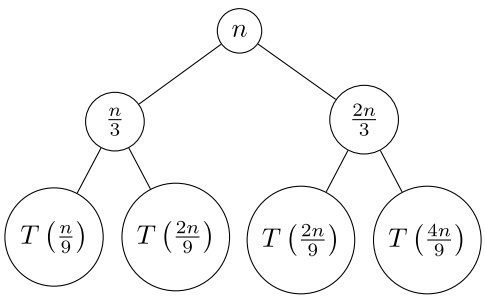
$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$$

a note to keep in mind is that the sum of all the nodes of the tree must always be equal to $T(n)$
with that in mind, the first step would be:



$$
\begin{aligned}
T\left(\frac{2n}{3}\right) &= T\left(\frac{2 \cdot \frac{2n}{3}}{3}\right) + T\left(\frac{\frac{2n}{3}}{3}\right) + \frac{2n}{3} = T\left(\frac{4n}{9}\right) + T\left(\frac{2n}{9}\right) + \frac{2n}{3} \\
T\left(\frac{n}{3}\right) &= T\left(\frac{2 \cdot \frac{n}{3}}{3}\right) + T\left(\frac{\frac{n}{3}}{3}\right) + \frac{n}{3} = T\left(\frac{2n}{9}\right) + T\left(\frac{n}{9}\right) + \frac{n}{3}
\end{aligned}
$$

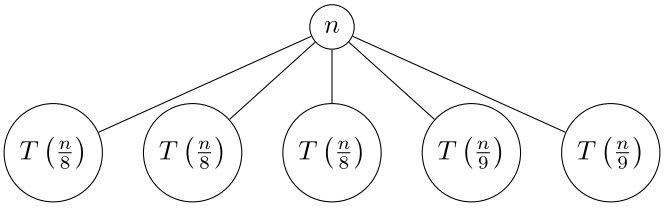
according to this, the tree with the new nodes would be:



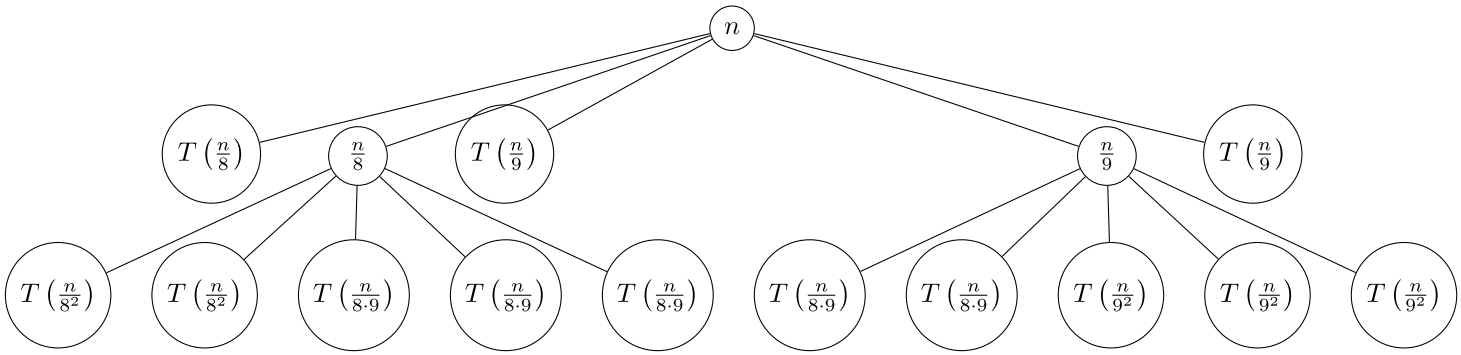
Example 3.0.2. consider the following recurrence relation

$$T(n) = 2T\left(\frac{n}{8}\right) + 3T\left(\frac{n}{9}\right) + n$$

we cant determine Θ using the Master Theorem, but we can using a recursion tree
the first step would be:



$$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{8^2}\right) + 3T\left(\frac{n}{8\cdot 9}\right) + \frac{n}{8}$$
$$T\left(\frac{n}{9}\right) = 2T\left(\frac{n}{8\cdot 9}\right) + 3T\left(\frac{n}{9^2}\right) + \frac{n}{9}$$



$$\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&= \text{sum of all nodes in full rows} \\
&\leq n + n\left(\frac{2}{8}+\frac{3}{9}\right) + n\left(\frac{2}{8}+\frac{3}{9}\right)^2 + \cdots + n\left(\frac{2}{8}+\frac{3}{9}\right)^{x-1} \\
&\leq n\left(\left(\frac{2}{8}+\frac{3}{9}\right) + \left(\frac{2}{8}+\frac{3}{9}\right)^2 + \cdots + \left(\frac{2}{8}+\frac{3}{9}\right)^{x-1}\right) \\
&= n \cdot \frac{1}{1-\left(\frac{2}{8}+\frac{3}{9}\right)} \\
&= \Theta(n)
\end{aligned}$$

note the sum was found using geometric progression formula
and so we have shown that $T(n) = O(n)$

$$T(n) = \text{sum of all nodes in the tree} \geq \text{root} = n = \Theta(n)$$

and so we have shown that $T(n) = \Omega(n)$
and so $T(n) = \Theta(n)$

note

Example 3.0.3. this need some correction as the sum of the nodes in the tree isnt exactly 2^{x-1}

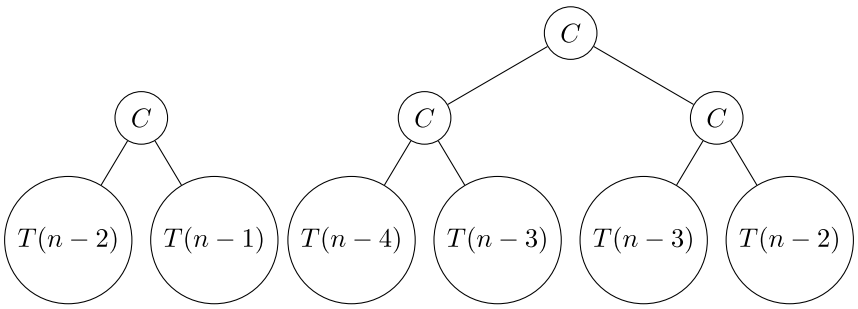
```
best_sum(A, i) {
  if i > size(A)
    return 0
  return max(best_sum(A, i+1), A[i] + best_sum(A, i+2))
}
```

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + C \\
&\leq 2T(n-1) + C \\
&\leq 2(2T(n-2) + C) + C = 4T(n-2) + 3C \\
&\leq 4(2T(n-3) + C) + 3C = 8T(n-3) + 7C \\
&\vdots \\
&\leq 2^{n-1}T(n-(n-1)) + \left(2^{n-1} - 1\right) C \\
&\leq 2^{n-1}T(1) + \left(2^{n-1} - 1\right) C \\
&\leq 2^n = \Theta(2^n)
\end{aligned}
$$

we proved $T(n) = O(2^n)$, now we try showing $T(n) = \Omega(2^n)$

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + C \\
&\geq 2T(n-2) + C \\
&\geq 2(2T(n-4) + C) + C = 4T(n-4) + 3C \\
&\geq 4(2T(n-6) + C) + 3C = 8T(n-6) + 7C \\
&\vdots \\
&\geq 2^{n/2}T(n-(n-2)) + \left(2^{n/2} - 1\right) C \\
&\geq 2^{n/2}T(2) + \left(2^{n/2} - 1\right) C \\
&\geq 2^{n/2} = \Theta\left(2^{n/2}\right)
\end{aligned}
$$

and so we showed that $T(n) = \Omega\left(2^{n/2}\right)$, which means there isnt a tight bound on the time complexity of this function
using a recursion tree:



$$
\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&= \text{sum of all nodes in all levels} \\
&\leq 2^{x-1} \\
&\leq 2^x = \Theta(2^x)
\end{aligned}
$$

we denote by y the last level with a full row, for this row to exist, $n-2(y-1) \geq 0 \implies n+2 \geq 2y \implies \frac{n+2}{2} \geq y$, so we have $\frac{n+2}{2}$ full rows

$$
\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&\geq \text{sum of full rows} \\
&\geq \sum_{i=1}^{\frac{n+2}{2}} 2^{i-1} \\
&\geq 2^{\frac{1}{2}n+1} - 1 \\
&\geq \frac{2^{n/2}}{2} - 1
\end{aligned}
$$

the division by 2 doesnt affect big omega so $T(n) = \Omega\left(2^{n/2}\right)$

4 master theorem

4.1 dividing functions

to solve a recurrence relation of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \text{ where } a \geq 1, b > 1$$

we compare $f(n)$ with $n^{\log_b a}$ and check which one dominates, the possible outcomes are:

$$\exists \epsilon > 0 \mid f(n) = O\left(n^{\log_b(a)-\epsilon}\right) \implies T(n) = \Theta\left(n^{\log_b a}\right) \tag{1}$$

$$f(n) = \Theta\left(n^{\log_b a}\right) \implies T(n) = \Theta\left(n^{\log_b a} \log n\right) \tag{2}$$

$$\exists \epsilon > 0 \mid f(n) = \Omega\left(n^{\log_b(a)+\epsilon}\right) \text{ and } 0 < c < 1 \;\middle|\; af\left(\frac{n}{b}\right) < cf(n) \implies T(n) = \Theta(f(n)) \tag{3}$$

Example 4.1.1. example of the first possibility

$$
\begin{gathered}
T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2 \\
a = 8,\ b = 2,\ f(n) = 1000n^2
\end{gathered}
$$

the conditions of the first possibility are met:

$$n^{\log_b a} = n^{\log_2 8} = n^3 = O\left(1000n^2\right)$$

therefore we get:

$$T(n) = \Theta(n^3)$$

Example 4.1.2. example of the second possibility

$$
\begin{gathered}
T(n) = 2T\left(\frac{n}{2}\right) + 10n \\
a = 2, b = 2, f(n) = 10n
\end{gathered}
$$

the conditions of the second possibility are met:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = \Theta(10n)$$

we get:

$$T(n) = \Theta(n \log n)$$

Example 4.1.3. example of the third possibility

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$
$$a = 2, b = 2, f(n) = n^2$$

the conditions of the third possibility are met:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = \Omega(n^2)$$

we get:

$$T(n) = \Theta\left(n^2\right)$$

4.2 decreasing functions

the **master theorem** can be used to solve recurrences of the form $T(n) = aT(n-b) + f(n)$, where $a \geq 1$ and $b > 0$ and $f(n)$ is **asymptotically positive**. (asymptotically positive means that the function is positive for all sufficiently large n.) this recurrence describes an algorithm that divides a problem of size n into sub problems, each of size $n - b$, and solves them recursively.
the theorem is as follows:
if $T(n) = aT(n-b) + f(n)$, where $a \geq 1$, $b > 0$, $f(n) = O\left(n^k\right)$ and $k \geq 0$
we consider 3 cases:
case 1, if $a = 1$

$$T(n) = O(n \cdot f(n)) \text{ or } O\left(n^{k+1}\right)$$

Example 4.2.1.

$$T(n) = T(n-1) + 1 \implies O(n) \tag{4}$$
$$T(n) = T(n-1) + n \implies O(n^2) \tag{5}$$
$$T(n) = T(n-1) + \log n \implies O(n \log n) \tag{6}$$

case 2, if $a > 1$

$$T(n) = O\left(a^{\frac{n}{b}} \cdot f(n)\right) \text{ or } O\left(a^{\frac{n}{b}} \cdot n^k\right)$$

Example 4.2.2.

$$T(n) = 2T(n-1) + 1 \implies O(2^n) \tag{7}$$
$$T(n) = 3T(n-1) + 1 \implies O(3^n) \tag{8}$$
$$T(n) = 2T(n-1) + n \implies O(n \cdot 2^n) \tag{9}$$

case 3, if $a < 1$

$$T(n) = O(f(n)) \text{ or } O\left(n^k\right)$$